

利用大语言模型提升智能家居中的个性化用户体验

Jordan Rey-Jouanchicot^{1,2,3}, André Bottaro³, Eric Campo², Jean-Léon Bouraoui³, Nadin Vigouroux¹, Frédéric Vella¹ IRIT, 图卢兹大学, CNRS, Toulouse
INP, UT3, UT2J, 图卢兹, 法国²LAAS-CNRS, 图卢兹大学, CNRS, UT2J, 图卢兹, 法国³Orange Innovation, Blagnac, 31700, 法国

摘要

智能家居自动化系统旨在提升用户在其居住环境中的舒适度和便利性。然而，使自动化适应用户需求仍是一项挑战。事实上，许多系统仍然依赖为每个智能对象手工编写的例行程序。

本文提出了一种原创的智能家居架构，利用大语言模型（LLMs）和用户偏好来突破家庭环境中个性化和直观性的边界。本文探讨了一种以人为中心的方法，利用大语言模型提供的通用知识来学习并促进与环境的交互。

所提出模型的优势在一组场景中得到了验证，并与各种LLM实现进行了比较分析。评估了一些指标，以确定系统维持舒适度、安全性和用户偏好的能力。本文详细介绍了实际部署与评估的方法。

所提出的使用偏好的方法在Starling 7B Alpha LLM上显示出平均分数最高提升52.3%，平均处理时间降低35.6%。此外，其性能比未使用偏好的较大模型好26.4%，处理时间几乎快20倍。

1 引言

设备网络被部署用于协助人类的日常活动，利用上下文和知识的概念来决定应采取的最佳动作 [Weiser, 1991] [Dey, 2001]。事实上，许多房屋都覆盖着无线和有线网络，并配备了电子设备，使居住者能够控制其环境，以实现舒适、娱乐、安防、能源管理和老年护理。

然而，智能家居自动化系统仍然未能实现在每种情况下自主采取最佳动作的目标。调整自动化例行程序以满足每一种需求

对于每一种家庭配置、每一组设备，无论是否可用、功能是否正常，仍然是一个挑战。事实上，当今大多数系统都只是为频繁发生的简单例行程序而配置的。例如，通过播放音乐、响铃、控制灯光、百叶窗、供暖、通风和空调，为起床或离家时间设置家庭场景。较少发生情况下的更精确需求并未被覆盖。此外，如果某个例行程序中缺少某些设备，就不会做出使用替代设备的决定。虽然人工智能可以在了解情境以及用户采取的相关动作方面发挥作用 [Rashidi and Cook, 2009]，它仍然需要时间来理解用户习惯，并且永远无法覆盖家庭中遇到的各种情境。

普适计算的主要技术挑战之一是，系统需要能够了解用户潜在需求的广泛多样性，并适应现有设备的各种功能，而这些设备和位置的配置可能千差万别。由于执行环境特别动态，应用程序需要感知其上下文并做出适当的动作。

大语言模型 [Radford 等人, 2019] 能够将这种通用知识立即赋予智能家居等应用。本质上，这些模型已经获取了海量知识。它们在海量多样化的文本来源上训练，从而能够覆盖各种主题、事实和概念，包括家用设备在广泛场景下为满足用户需求而应有的预期动作。

检索增强生成 [Lewis 等人, 2020] 是一种通过让大语言模型获取更有针对性和更精确的信息来提高其准确性的技术。其实现方式是使用检索组件搜索特定数据库，并将检索到的信息连同用户的查询一起输入模型。

本文提出了一种软件架构，将当今可用的一般大语言模型知识集成到智能家居自动化系统中。大语言模型被置于家庭决策系统的核心，并参与对每个事件的反应，以推断出下一步最佳行动。本文研究了在智能家居自动化中结合大语言模型与偏好的方式，其贡献还包括以用户为中心的智能家居状态和动作的自然语言表示。最后，使用多种大语言模型和不同提示风格在智能家居中进行了决策实验。

下一节介绍文献中的相关工作。然后，详细阐述了所提出的软件架构，特别是将大语言模型用于决策，并结合检索增强生成进行用户上下文检索。随后，实验部分描述了一个专门针对大语言模型和提示风格的基准测试，使用一组家庭场景，从而得出下一节中的结果。对这些结果进行了讨论，最后一节则致力于结论和未来工作。

2 相关工作

这一主题较新，目前关于大语言模型在智能家居中应用的论文还很少。以下论文因其在智能家居自动化系统决策中利用用户偏好知识的方法而被选中。本节无法涵盖这一更广泛领域的全部内容。

[Oliveira 等人, 2022] 提出了一个带有信念-愿望-意图认知模型的多智能体环境，以在智能家居环境中透明地支持自适应性和偏好。任何可能的交互都可以通过这种方式建模，以允许提出替代方案，但这需要在语义表示方面进行大量工作才能完全完整。

另一篇论文利用通用知识信息来提高系统对偏好的自适应性。[Ruiz等人, 2021] 提出了一种方法，使用三个基于知识的系统：一个包含通用知识，一个包含技能知识，一个包含上下文信息（如设备位置），并基于所有这些信息为中间件平台生成规则模型。

这种方法需要非常严格的语义建模来处理大多数场景，并且无法利用上下文中隐式给出的某些信息。

[Shuvo, 2022] 提出了一种基于actor-to-critic (A2C) 的算法，适用于智能家居中能源消耗的决策。在这项工作中，每一步都对每个设备应用A2C算法，以选择最佳动作，并使用当时的活动和电价作为输入。

一个关键因素是，每个家电的动作集取决于与该家电关联的类别，从而为模型添加初始知识，以确保动作基于家电的重要性。该系统需要学习，设备可用性中的任何导致最优决策不同的变化，都需要经过大量步骤才能适应。

此外，上下文越大，每个场景就越难收敛，以学习最合适的设备状态。论文HESIDR, [Zhang 等人, 2023] 还提出了一个用于智能家居能源消耗的系统，它结合了一组控制规则和强化学习来减少适应时间。

[Peng 等人提出的研究, 2020] 描述了一种使用深度强化学习进行家庭自动化决策的方法。它展示了学习何时开灯的能力，但支持的上下文非常有限，因此该应用仅限于学习按15分钟的时间表开灯，而这一时间间隔对于该应用来说相当长。

[King 等人, 2023a; King等人, 2023b] 提出了第一个

智能家居自动化的方法，使用来自智能家居中间件平台的JSON数据表示，并通过实验使用大语言模型根据用户请求选择动作。

这一方法是朝着利用这些模型所提供通用知识迈出的第一步。然而，它不支持用户偏好，其思路是根据初始用户请求来选择动作，而非主动选择。这些工作提出了管理决策的方法，但受所支持的上下文数据限制。使用符号AI方法的工作可以展现出很强的适应性，但代价是需要大量的语义建模，并且需要使其能够适应未来偏好的变化。因此，提出了一种在支持上下文数据的同时加入偏好的新方法。

3 提议架构

本研究提出了一种用于智能家居自动化系统决策的新架构。该系统使用大语言模型，并提出了添加用户偏好的方法，以便根据上下文和用户选择动作。其目标是成为一个主动系统。在家庭中发生的每个事件中，系统都会对设备提出动作，以使家庭状态与用户需求和偏好保持一致。得益于大语言模型在生成基于设备状态和动作列表的家庭文本表示的同时处理数据的能力，该系统支持不同类型的数据。

所提出的系统旨在能够适应随时间的用户偏好变化。实际上，大语言模型无需重新训练，并且能够处理设备配置变更、基本不可用或设备故障。

这一原因直接导致使用检索增强生成，或将知识直接注入提示词，以便在人工智能每次执行时检索最新偏好，而不是进行微调。实际上，对偏好进行微调并不现实，因为每次用户偏好发生变化都需要新的迭代，这将成本高昂且计算开销巨大。

实现了一个家庭模拟器，它获取传感器配置信息并生成文本表示；该家庭模拟器还被用于根据其数据生成动作列表。

本节详细介绍为所提出的系统架构设计的主要组件。图1展示了该架构，包括家庭模拟器。

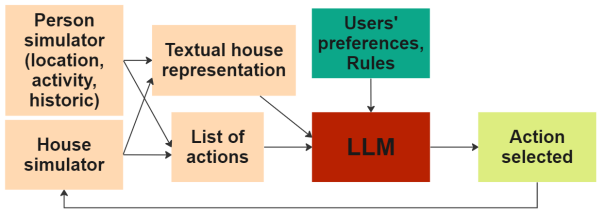


图1：所实现的架构

系统生成一个以用户为中心的文本描述，描述该

家庭和动作列表，并可控制已连接设备。它将为每种情况筛选出可采取的相关动作。例如，它会限制一些可能被禁止的动作，以确保用户安全。模型会使用该动作列表来选择最优动作。

关于上下文数据表示编辑：

- 用户位置将与其当前活动和先前活动历史一同列出。

- 支持住宅内执行的先前动作历史。

- 所有房间、传感器和执行器均通过其名称展示，这些名称由用户可选地提供。

- 一些传感器和执行器可提供更全局的数据和控制能力，例如内部与外部温度传感器、气体传感器、湿度传感器、HVAC系统等。

该实现支持一些具有专用自然表示的设备类别，针对某些类型的数据生成更自然的语句：灯光、二氧化碳传感器、智能窗帘等。它还可以通过通用表示模板支持任何附加数据传感器，利用智能家居环境中的设备名称和数据状态。用户添加的有意义的命名有助于系统理解设备使用情况。

动作提议算法1大幅缩减了可能动作集。它假设只有位于房间内或全局的设备才能被打开，但所有已开启的设备都可以被关闭。就可能动作列表而言，其思路是根据某些条件和设备类型过滤所支持的动作。这种方法之所以能够实现，是因为大语言模型原生支持改变输出动作空间，而无需训练。

算法1 动态设备：动作构建算法

要求： *userid, devices list*

```
1: 对于所有 device name, device kind, device location,  
   device state 在 devices list 中执行  
2:   如果 device kind = "actuator", 则  
3:     如果 user location[userid] = device location 或  
       device state = 1 则  
4:       devices.append(device name 是 device state)  
5:       action-vector.append(1)  
6:     结束如果  
7:   结束如果  
8: 结束循环  
9: devices.append(" 与用户交互" )  
10: action-vector.append(2)  
11: devices.append(" 无需动作" )  
12: action-vector.append(0)  
13: 返回 (action vector, devices)
```

任何类型的设备都可以轻松支持：只需将其添加到表示中，大语言模型便会凭借其内部知识处理这些数据。这种知识使大语言模型能够获取包含有偏语境数据的家庭自然人类描述，这与许多传统的家庭自动化系统不同。后者无法利用

灯光或房间名称等信息。这些数据通过某种提示风格传递给大语言模型，提示即调用大语言模型并附带上下文参数的方式。不同的提示风格将在下一节中描述。

所有提示风格的一个共同目标是：利用该大语言模型提供的关于整个世界的知识，来处理表示中添加的变化甚至新型传感器。

用户偏好与规则模块是一个数据库，包含系统基本规则、关于人类偏好的一般性知识以及特定用户偏好的信息。

为进行评估，使用预定义场景建立了一个基准测试。

关于大语言模型，选择了一些现成模型，并配备本地推理引擎后端。

4 实验

为实验定义了不同的目标。

- 评估通过添加用户偏好所带来的改进，并采用各种技术来实现这一目标。
- 评估自然语言表示智能家居自动化状态相较于JSON表示所带来的改进，因为大语言模型是在自然语言语料库上训练的，即使其中包含代码等其他类型的数据。

为与实验目标保持一致，这些评估使用了不同的指标：

- 评分：根据表1中定义的评分值，通过执行获得每个模型和每种提示风格在各场景中的评分。
- 处理时间：总运行时间，包括上下文数据和动作列表表示的构建、使用提示风格进行的推理、如果提示风格使用检索增强生成则包括检索增强生成的使用，以及格式化的大语言模型响应的处理。

十一个评估场景被定义为起点，并预定义了可接受动作及其具体评分。表1按名称列出了这十一个场景及相关的奖励值。每个场景都关联一个类别，目标是根据主要被评估能力的名称对场景进行归类。

图2展示了场景1中生成的上下文表示（文本）传输给大语言模型的示例。

偏好与规则数据库在单个文件中为所有场景定义。这些数据是自然书写的句子，在每个句子末尾记录有关风格的信息：规则、偏好、通用性。其想法是通过关键词向大语言模型传达每条数据的重要性。通用性被认为最不重要，偏好其次，规则最为重要。该数据库包含一些偏好、通用性以及一些规则。它旨在处理某些场景，帮助其他一些场景，但不能为所有场景提供解决方案。数据被输入向量数据库，以便可以使用检索增强生成而不是提示词来将这些数据传递给大语言模型。

场景 Name	年级	关联答案	类别
下床 在夜间	2 1 0	打开辅助灯或主灯且 降低亮度水平 打开主灯 其他所有	安全性
观看 电视：晚 傍晚	2 1 0	打开辅助灯或主灯，并 降低亮度 打开主灯，打开窗帘，讨论 其他所有	舒适度
从...出来 床铺问题 与二氧化碳	2 1 0	告知用户风险 执行一个动作并告知用户风险 其他所有	安全性
返回 上床睡觉在 夜间	2 1 0	打开辅助灯或主灯，使用 降低亮度水平 打开主灯 其他所有	安全性
傍晚 睡眠中： 电视开机	2 1 0	关闭电视 关闭所有打开的 其他所有	偏好
用餐时 观看 TV	2 1 0	打开辅助灯或主灯且 降低亮度水平，打开窗帘 关闭主灯，不做任何操作 其他所有	偏好
忘记 关闭电视： 用户外出	2 1 0	关闭电视，关闭暖通空调 关闭所有灯光 其他所有	舒适度
太低 温度	2 1 0	打开暖通空调 打开窗帘 其他所有	偏好
Low 亮度 day	2 1 0	打开窗帘 打开房间里的任何一盏灯 其他所有	偏好
失败 窗帘	2 1 0	打开房间里的任何一盏灯 打开窗帘 其他所有	舒适度
忘记 关闭 灯光	2 1 0	关闭任何灯光、所有灯光或暖通空调 其他所有	偏好

表1：场景响应及其评分、关联答案和评估类别。

针对所有场景比较了四种不同的提示风格：

- 直接：一个系统提示， 以及一个用于请求以指定格式给出直接答案的提示。
- directPref：包含来自数据库的偏好、规则和通用性的系统提示， 以及一个用于请求以指定格式给出答案的提示。
- OpenQuestion：两步链：一个系统提示， 以及一个请求“3个主要问题的列表”的提示。对于3个问题中的每一个，使用检索增强生成获取3个最接近的偏好。一个用于请求以指定格式给出答案的提示。
- ThreeQuestion：三步链：一个系统提示， 一个请求“3个主要问题的列表”的提示。对于3个问题中的每一个，使用检索增强生成获取3个最接近的偏好。一个用于请求以指定格式给出答案的提示（请求2次）。最后一个提示用于基于之前的答案以指定格式请求答案。

数据库和提示风格可在补充材料中找到。

所有提示风格的一个共同点是输出中预期的动作：如上所述，大多数支持的设备（如灯光或HVAC系统）在动作列表中被视为开关；因此，在输出中，

除了“推理”和“动作”键之外，所有提示风格都提供了三个可选键：温度、亮度和解释。这使模型能够在执行相关动作时分别调整HVAC系统的温度、通过调光改变灯的亮度，或提供解释以向用户传递一条句子。

实现了两种表示房屋状态数据的方式，两者均使用相同的输入数据：

- JSON: JSON表示
 - 文本: 一种完全自然的文本表示
- 该系统的实现使用了各种开源大语言模型进行评估，包括：

- Starling Alpha 7B [Zhu 等人, 2023]- 8bpw
- Qwen 1.5 14B [Bai 等人, 2023]- 5bpw
- Qwen 1.5 72B [Bai 等人, 2023]- 3.5bpw

这三个模型因其性能以及覆盖三种主要开源模型大小而被选中。它们被用于评估所提出的提示方法和数据表示的影响。

Qwen 1.5 72B 拥有约720亿个参数，是目前可用的最佳开源模型之一。Starling 7B Alpha 拥有约70亿个参数，是一款出色的较小模型，基于 Mistral 7B，后者在其规模上在各种基准测试中表现高效。Qwen 1.5 14B 模型是 Qwen 1.5 72B 的小型版本，被选中作为中间模型添加。

所有这些模型都使用了经过量化的版本——¹，一种用于减少推理时间和内存占用的技术，为每个模型选择的量化以每权重比特数（bpw）给出。经过量化，它们分别需要大约8GB、12GB和44GB的内存。

每个模型均在本地实例上进行评估，并通过基于引擎的API 后端在本地提供服务，使用TabbyAPI²（基于ExLLamaV2¹），支持多GPU且不进行自动分割，并使用配备Ryzen 9 7950x、96GB DDR5内存（运行频率为5600MHz）和2块Nvidia RTX 4090（每块配备24GB专用内存）的工作站，运行Ubuntu 23.10。

实验事先在各种非受控场景中进行，以确定大语言模型参数。唯一目的是降低从一个周期到下一个周期的非确定性，因此最终从默认引擎参数修改而来的参数如下：

- 最大令牌数，即输出中的最大令牌数量：300
- 最小p值，即令牌必须达到的最小百分比值才会被考虑（该值根据最大令牌概率进行缩放）：0.05
- 温度，调节随机性的参数：0.2

检索增强生成是使用 Langchain[Topsakal 和 Akinci 实现的, 2023] 并使用来自 HuggingFace 的推理引擎³，

¹ <https://github.com/turboderp/exllamav2>

² <https://github.com/theroyallab/tabbyAPI>

³ <https://github.com/huggingface/text-embeddings-inference>

以在本地运行一个嵌入模型：BAAI/bge-large-en-v1.5[Xiao 等人, 2023], 并将一个 Elasticsearch⁴ 本地实例用作向量数据库, 这两个实例均在 CPU 及相关内存上运行。

为了评估系统性能, 每个场景使用每种提示风格执行10次, 并对每个响应给出评分。随后, 结果按总体统计每个提示类别的总分, 并统计与每个问题相关的每个已定义指标的分数。此外, 还会测量系统的完整处理时间, 以估算不同提示风格的平均延迟。

每个场景的理论随机行动评分使用以下公式1计算, 并作为基线。

$$\text{评分}_s = \frac{\text{评分为1的动作数}_s + 2 \times \text{评分为2的动作数}_s}{\text{动作数}_s} \quad (1)$$

图2展示了使用所提出的自然语言文本表示方法的场景数据表示示例。

当前房屋状态: 用户1在客厅。用户正在观看电视。之前: 用户刚才正在看电视。客厅: 窗帘关闭。灯光: 主灯、落地灯均关闭。房间内有一台电视, 其状态为开启。室内二氧化碳浓度为513ppm。…… 厨房: 窗帘关闭。灯光: 主灯关闭。室内二氧化碳浓度为473ppm。房屋今天已清洁。预计每周清洁一次。集中式暖通空调系统已开启, 目标为 20°C。入口智能门已上锁。时间: 晚上10:21。全屋温度为 20°C, 室外温度为 5°C。

图2: 场景“夜间起床”的自然数据表示片段

如前所述, 表示中包含了几种数据类型: 数值、布尔值和字符串。使用大语言模型处理上下文, 使系统能够支持所有这些数据类型。

关于行动列表构建算法, 例如在其中一个场景中, 它将动作集从18个以上减少到6个动作, 在这种情况下相当于将动作集缩减为原来的约三分之一。这个缩减后的动作集考虑了照明设备是基本开关、无法设置不同亮度级别这一事实。

这个有限的动作集使得使用一个从动作集中随机选择一个动作的模型作为基线变得有趣。在缩小的动作空间下, 随机答案可能也不会太差。

5 结果

5.1 数据表示

结果的第一项分析关注的是使用自然表示相对于JSON表示的优势。

模型	JSON	自然
Qwen 1.5 72B	1.25	1.18
Qwen 1.5 14B	0.63	0.99
Starling Alpha 7B	1.03	1.18

表 2: 比较两种上下文表示: 每个场景每次执行中模型的平均分数

表2展示了使用JSON表示与更自然的表示所得结果之间的平均差异。可以看出, 较大的模型几乎不受上下文表示影响而保持稳定, 平均而言, 使用JSON表示的效果甚至略好, 约高出5.9%。然而, 在较小的模型上, 自然语言表示大幅提升了性能: 使用Starling 7B时平均性能提高14.6%, 使用Qwen 1.5 14B时提高57.1%。在处理时间方面, 两种表示下各模型的平均结果非常相近。总体而言, 所有模型的平均性能提升了21.9%, 尽管在较大的Qwen模型上结果并非如此, 但这使自然表示更具效率。

5.2 用户偏好

图3展示了所选3个模型在4种不同提示风格和2种不同表示类型下对场景的响应所获得的平均分数。

它首先表明, Qwen 72B模型比另外两个模型稳定得多, 且提示风格对响应质量的影响并不那么大。至于两个较小的模型, 可以注意到结果的不一致性较大, 且随着所选择的提示风格而变化, 尤其是Qwen 14B模型。

大语言模型的结果与对应于随机选择动作的基线进行了比较。平均而言, 这些结果仍落后于任何没有偏好的模型(比使用“直接”提示风格的最差结果低37.1%)。然而, 由于该算法缩减了动作集, 且每个场景下多个响应都是可接受的, 随机基线获得的评分有时会优于某些使用大语言模型的实验的结果。

多问题序列要求模型保持一致并遵循预期指令格式。此外, 由于仅使用提示工程来确保格式, 某些带有多个问题的提示风格可能导致无效响应, 从而迫使模型在提议的设置中采取默认动作。该默认动作被设置为不执行任何操作, 并告知用户它未能执行操作。这降低了某些模型在某些提示风格下的性能。在图3中, 失败率衡量的是无效响应的比率。Qwen 14B的结果尤其低于基线, 因为它们在大量场景中未能作答。

⁴<https://github.com/elastic/elasticsearch>

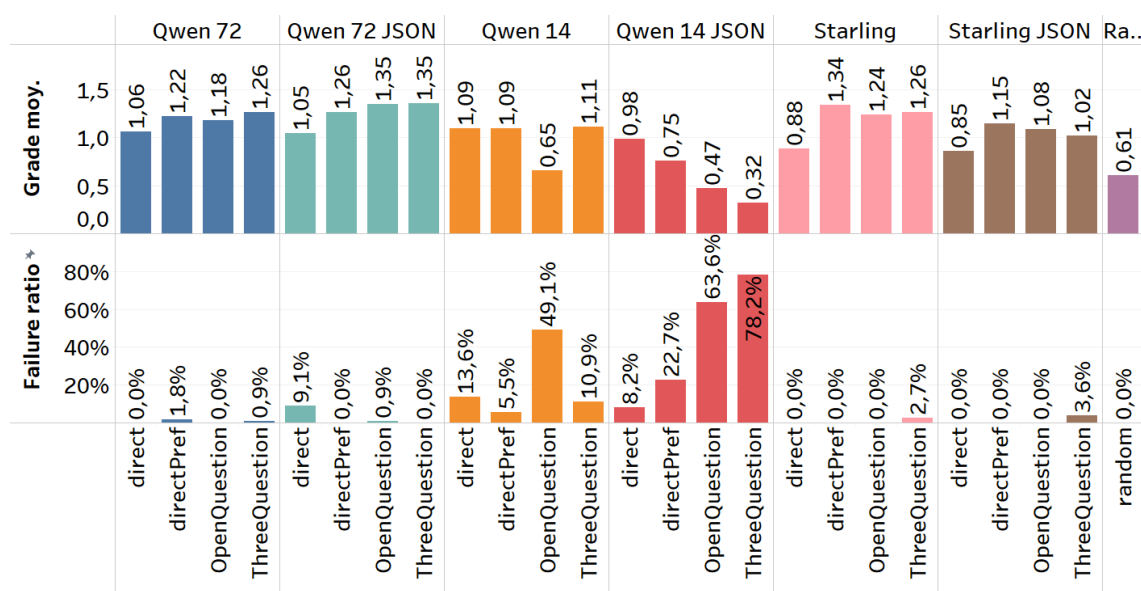


图3：按模型、数据表示和提示风格划分的平均分数

平均而言，“directPref”提示模型相对于采用自然表示的基础提示获得了11.3%的提升，而在使用JSON表示时甚至提升了20.0%。

对于“OpenQuestion”和“ThreeQuestion”，使用JSON表示可获得最佳结果，比“直接”提示风格提升了28.6%。

使用自然表示时，Qwen 14B模型在除“OpenQuestion”外的所有提示风格下都获得了几乎稳定的结果；与“OpenQuestion”的主要差异似乎与高失败率有关，如图3所示。使用JSON表示时，所有高级提示风格的失败率都非常高，以至于最佳结果由最基础的提示取得。在这种情况下，“直接”提示风格比“directPref”提示风格效率高出30.7%。

在Starling 7B上，两种表示下的最佳结果均通过“directPref”获得；在Starling模型上，其结果比直接提示风格高52.3%，并分别比“OpenQuestion”和“ThreeQuestion”提示风格高8.1%和6.3%。

此外，在Starling 7B Alpha上，使用“directPref”提示风格而非“direct”，平均处理时间减少了35.6%。

使用JSON表示时，平均分数结果呈现相同趋势，但总体值较低。

表3显示，基于RAG的提示模型可以比包含所有数据的单个提示词产生更好的结果，正如Qwen 72B所示。然而，到目前为止，大语言模型推理仍然较慢，使用复杂提示词会导致较小模型在测试场景中的准确率下降。

将推理时间也纳入权衡后，表3突出显示了采用“directPref”的Starling 7B Alpha的结果。它几乎优于所有其他模型，除了采用JSON表的Qwen 72B

模型	提示风格	平均分数	处理时间（秒）
Qwen 72	direct	1.06	9.04
	directPref	1.22	7.01
	OpenQuestion	1.18	24.43
	ThreeQuestion	1.26	42.13
Qwen 72 JSON	直接	1.05	9.02
	directPref	1.26	6.71
	OpenQuestion	1.35	25.20
	ThreeQuestion	1.35	41.47
Qwen 14	直接	1.09	1.22
	directPref	1.09	1.16
	OpenQuestion	0.65	8.83
	ThreeQuestion	1.11	15.54
Qwen 14 JSON	直接	0.98	1.28
	directPref	0.75	1.21
	OpenQuestion	0.47	9.39
	ThreeQuestion	0.32	16.62
Starling	直接	0.88	0.73
	directPref	1.34	0.47
	OpenQuestion	1.24	3.98
	ThreeQuestion	1.26	6.23
Starling JSON	直接	0.85	0.48
	directPref	1.15	0.48
	OpenQuestion	1.08	3.77
	ThreeQuestion	1.02	6.20

表3：每种提示风格与模型的平均分数比较及推理时间

示和提示链外，但它的速度却快53.6倍，且仅低0.7%。

对于Qwen 72B，使用更复杂的模型（OpenQuestion、ThreeQuestion）可以带来更好的结果，正如之前特别是在JSON表示中所见，但这需要付出代价：推理时间。Qwen 72B模型由于其参数数量以及生成单个token所需的大量操作，已经明显更慢。例如，使用“directPref”进行推理平均需要6.86秒（JSON和文本表示），而使用“OpenQuestion”（比“ThreeQuestion”快约60%）则慢3.6倍。

在当前这种硬件的状态下，不可能

将它们视为管理智能家居自动化系统的可行替代方案，以这样的反应时间。

6 讨论

本节主要讨论所选大语言模型和提示技术的结果，以便为实际实验做出选择。本研究既强调了自身的优势，也指出了由此引发的新挑战。

如前所述，使用更大的大语言模型（如 Qwen 72B）可以在无偏好场景中提供更高的稳定性。事实上，在这种情况下，Qwen 72 比 Starling 7B 好 19.9%。然而，也存在缺点：首先是如前所述的推理时间，其次是所需的硬件基础设施。Qwen 1.5 72B 的量化版本需要 44GB 内存，而采用较低量化的 Starling 7B Alpha 大约只需 8GB。

与使用 JSON 表示的 Qwen 72B 模型相比，采用“directPref”的 Starling 7B Alpha 利用了自然表示，并通过复杂得多的提示技术实现了几乎相近的性能。这使得将 Starling 7B Alpha 与该提示风格结合使用成为未来工作的良好选择。它在评分方面表现出相似的性能，并且推理时间相对较低（平均：0.47 秒）。

此外，与采用自然表示且没有偏好的 Qwen 1.5 72B 相比，Starling 7B Alpha 使用“directPref”的性能提升了 26.4%，处理时间几乎快了 20 倍。

结果表明，遵循偏好是有优势的。然而，随着采用 RAG 的“OpenQuestion”和“ThreeQuestion”提示风格带来额外的平均计算时间，缺点也显现出来。在大多数情况下，使用 RAG 并没有带来优势。这无疑是由于数据库相对较小。如果系统需要更大的偏好与规则数据库，结果可能会不同，因为无法通过“directPref”提示风格直接提供这些数据。基于当前结果，对于需要更大数据库的用例，最佳选择仍然是采用“OpenQuestion”提示风格和自然表示的 Starling 7B Alpha，因为它提供的结果与用户偏好一致，平均分数仅比 Qwen 1.5 72B 低 8.1%，同时保持了可接受的平均推理时间（3.98 秒对比 25.20 秒）。

使用较小的模型会带来强制输出格式的挑战。最近已有新方法被提出，例如 Outlines⁵，以强化输出语法。应尽快对这些方法进行测试，以分析其推理时间成本以及对本文论结果的改进。

另一种方法是微调。先前已提到使用微调来处理用户偏好的局限性。然而，在智能家居领域数据上进行微调，可以让系统在改进上下文数据分析的同时，确保输出格式。

尽管所提出的系统因其全新的方法而无法与其他工作相比，但它既具有自身固有的优势，也存在缺点。首先，它不是确定性的。每种配置都运行了10次，但有些结果每次都不相同，这意味着系统的结果无法得到验证，必须开发一个安全层来保护某些动作。其次，大语言模型查询速度缓慢，且需要大量的计算和内存资源：无需训练的适应性是有代价的。

7 结论

本文提出了一种用于智能家居自动化系统的新架构，利用大语言模型与用户偏好来增强个性化用户体验。该方法借助大语言模型提供的通用知识，并将其与自然书写的规则和偏好相结合，以做出符合用户偏好的情境相关决策。该架构具有主动性，能够适应环境中的任何变化，这得益于大语言模型所提供的鲁棒性。

以用户为中心的动作列表构建器利用这种能力来逐步减少动作集，因为环境表示也利用了用户为设备赋予的名称，以更好地支持用户。

实验结果表明，与不包含用户偏好的实现相比，该架构在改善与用户偏好对齐方面展现出潜力，性能提升最高可达52.2%。

研究表明，特别是对于小型模型，使用自然表示而非JSON表示会带来性能提升，平均提升21.9%。

尽管该系统在一组已定义场景中展现出有前景的结果，但由于随机行为以及相比传统机器学习方法更慢的推理时间，它也带来了挑战。这些缺点被系统无需重新训练即可动态适应偏好、家电和家庭配置变化的能力所抵消。

未来工作将专注于在真实世界智能家居中间件系统（如 OpenHAB[Portal'es 等 2019]）中实现该系统，以评估其在真实用户中的表现。将提出相应机制，使用户能够自然地添加和移除偏好，并探索这些规则和偏好自动演化的可能性。

⁵<https://github.com/outlines-dev/outlines>

伦理声明

不存在伦理问题。

参考文献

- [Bai等人, 2023] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou 和 Tianhang Zhu. Qwen技术报告. *arXiv预印本 arXiv:2309.16609*, 2023年.[Dey, 2001] Anind K. Dey. 理解并使用上下文. 个人与普适计算, 5(1):4–7, 2001年2月.[King等人, 2023a.] Evan King, Haoxiang Yu, Sangsu Lee 和 Christine Julien. “为派对做好准备”: 借助大语言模型探索更智能的智能空间. 预印本, 2023年. 发布方: arXiv, 版本号: 1.[King等人, 2023b.] Evan King, Haoxiang Yu, Sangsu Lee 和 Christine Julien. Sasha: 在智能家居中利用大语言模型进行创造性目标导向推理. 预印本, 2023年. 发布方: arXiv, 版本号: 1.[Lewis等人, 2020] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel 和 Douwe Kiela. 面向知识密集型NLP任务的检索增强生成. 见 第34届国际神经信息处理系统会议论文集, NIPS’ 20, 美国纽约州红钩, 2020年. Curran Associates Inc.[Oliveira等人, 2022] Pedro Filipe Oliveira, Paulo Novais 和 Paulo Matos. 用于管理舒适偏好与冲突的自适应环境系统. 见 Ana I. Pereira, Andrej Kosić, Florbela P. Fernandes, Maria F. Pacheco, João P. Teixeira 和 Rui P. Lopes 编, 《优化、学习算法与应用》, 第685–700页, Cham, 2022年. Springer International Publishing.[Peng等人, 2020] Z. Peng, X. Li 和 F. Yan. 智能家居自主系统的自适应深度学习模型. 见2020年智能交通、大数据与智慧城市国际会议 (ICITBS), 第707–710页, 美国加利福尼亚州洛斯阿拉米托斯, 2020年1月. IEEE计算机学会.[Portalés等人, 2019] Cristina Portalés, Sergio Casas 和 Kai Kreuzer. 家庭自动化的挑战与趋势: 使用开源平台 OpenHAB解决互操作性问题. 利用万物互联 (IoE) 加速创新机遇, 第148–174页, 2019年.[Radford等人, 2019] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei 和 Ilya Sutskever. 语言模型是无监督多任务学习器. *OpenAI博客*, 2019年.[Rashidi和Cook, 2009] P. Rashidi 和 D.J. Cook. 让居民保持在环: 使智能家居适应用户. *IEEE系统、人与控制论汇刊 - A部分: 系统与人类*, 39(5):949–959, 2009年9月.[Ruiz等人, 2021] Ana Rubio Ruiz, David Villa Alises, Rubén Cantarero Navarro, María J. Santofimia Romero, Javier Dorado Chaparro 和 Juan C. López López. 利用常识推理迈向更智能的智能家居. *Procedia计算机科学*, 192:666–675, 2021年.[Shuvo, 2022] Salman Sadiq Shuvo. 家庭能源推荐系统 (HERS): 一种基于居民反馈和活动的深度强化学习方法. *IEEE智能电网汇刊*, 13(4), 2022年7月.[Topsakal和Akinci, 2023] Oguzhan Topsakal 和 Tahir Cetin Akinci. 利用Langchain创建大语言模型应用: 快速开发LLM应用入门. 见应用工程与自然科学国际会议, 第1卷, 第1050–1056页, 2023年.[Weiser, 1991] Mark Weiser. 21世纪的计算机. *科学美国人*, 265(3):66–75, 1991年1月.[Xiao等人, 2023] Shitao Xiao, Zheng Liu, Peitian Zhang 和 Niklas Muennighoff. C-pack: 推进通用中文嵌入的打包资源, 2023年.[Zhang等人, 2023] Zhiyao Zhang, Yongxin Su, Mao Tan 和 Rui Cao. 融合领域知识和强化学习实现家庭集成需求响应在线优化. *人工智能的工程应用*, 121:105995, 2023年.[Zhu等人, 2023] Banghua Zhu, Evan Frick, Tianhao Wu, Hanlin Zhu 和 Jiantao Jiao. Starling-7b: 通过rlaif提升LLM有益性和无害性, 2023年11月.